

CLOUD COMPUTING TECHNOLOGY & ARCHITECTURES

Examination Notes — Unit 4 & Unit 5

Course Code: AI362IA / 21AI54 | Department of AI & ML

UNIT 4 — ENTERPRISE CLOUD ARCHITECTURE & MULTI-CLOUD PRINCIPLES

1. Introducing the Scaffold for Multi-Cloud Environments

PYQ / EXPECTED QUESTIONS

- Define the term 'Scaffold' in the context of multi-cloud environments. [CIE Improvement 2025, 2M]
- Develop a comprehensive strategy for cloud utilization in a higher education institution [SEE Aug 2025, 8M]
- Evaluate the advantages and potential challenges enterprises face when implementing a multi-cloud strategy [SEE 7a, 8M]
- Outline a strategic framework for deploying a multi-cloud environment [SEE 8b, 8M]

KEYWORDS

Scaffold | **Landing Zone** | **Cloud Adoption Framework** | **Well-Architected Framework** | **Hyperautomation** | **Guardrails** | **Multi-Cloud** | **Vendor Lock-in**

DEFINITIONS

Scaffold: A structure that supports the construction and maintenance of buildings — adopted by Microsoft to support, build, and manage environments deployed in Azure. In cloud, it is the foundation structure used to build and manage environments in a cloud landing zone.

Multi-Cloud: The use of two or more cloud computing systems at the same time. Deployments may use public, private, or a combination of clouds. Goal: redundancy and avoiding vendor lock-in.

Landing Zone: The configured platform (cloud environment) where applications and data are hosted, managed, and governed.

Well-Architected Framework: Best practice guidance provided by each major cloud provider (AWS, Azure, GCP) to help customers build secure, reliable, and cost-efficient environments.

EXPLANATION

Before deploying to any cloud, organizations must have a structured plan — just as you would not build a data center without a blueprint. The scaffold provides this foundational structure.

Key takeaway: You are building a data center. You are building it using public clouds, but it is a data center. Treat it as a data center.

Cloud adoption frameworks (from Azure, AWS, GCP) share common principles: cost control, governance, security, and identity management. The main challenge is a single dashboard for multi-cloud — addressed by hyperautomation and AIOps.

IMPORTANT POINTS

- Scaffold = foundation to build and manage cloud environments (landing zone)
- All major cloud providers issue Cloud Adoption Frameworks with similar ground principles
- The ultimate goal: one source of automation across platforms (hyperautomation)
- Scaffold comprises pillars: Governance, Identity, Cost Management, Security, and Operations

DIAGRAM AVAILABLE

(Refer Online: Multi-Cloud Scaffold Architecture)

2. Understanding Identities and Roles in the Cloud

PYQ / EXPECTED QUESTIONS

- List any two roles of Azure. [SEE Aug 2025, 2M]
- What are the key business drivers for setting a multi-cloud strategy? Explain the role of TOGAF ADM. [CIE Improvement 2025, 5M]

KEYWORDS

Active Directory (AD) | Authentication | Authorization | RBAC | IAM | LDAP | Kerberos | Azure AD | Federation | Zero Trust | B2B

DEFINITIONS

Authentication: The process of verifying identity — recognizing who a person or system is (via AD using Kerberos).

Authorization: Defining what an authenticated identity is allowed to do in the environment — managed via RBAC.

RBAC (Role-Based Access Control): A method to manage access based on roles. In Azure = RBAC; in AWS = IAM; in GCP = Cloud Identity.

Federation: Connecting AD with cloud identity providers so AD remains the single source of truth for identity management.

Zero Trust: A security principle: organizations do not trust anything within their networks. Every device, application, or user is monitored. Platforms are micro-segmented.

EXPLANATION

Everything in the cloud has an identity — human users, systems, and applications. The two tasks for any identity are:

- Authenticate: verify who the identity is (via Active Directory + Kerberos)
- Authorize: define what the identity can do (via RBAC/IAM)

Active Directory (AD) is the central identity store. Resources in cloud must be domain-joined. All cloud providers support AD, LDAP, and Kerberos.

For external identities (B2B): Azure uses Azure AD B2B, AWS uses Cognito, GCP uses Cloud Identity. AD always remains the single source of truth for identity management — cloud ADs (Azure AD) federate against it.

IMPORTANT POINTS

- AD = single source of truth for identity management across all clouds
- RBAC controls what identities can do; Authentication confirms who they are
- Zero Trust = assume the organization has been attacked; minimize blast radius via micro-segmentation
- Cloud providers: Azure AD (Azure), Cognito (AWS), Cloud Identity (GCP) for external/B2B identities

3. Enterprise Cloud Architecture

PYQ / EXPECTED QUESTIONS

- Illustrate with neat diagram the Cloud Computing Reference Model [SEE Aug 2025, 8M]
- What is the role of the 'Architecture Vision' phase in TOGAF's ADM cycle? [CIE Improvement 2025, 2M]
- What are the key business drivers for setting a multi-cloud strategy? [CIE Improvement 2025, 5M]

KEYWORDS

TOGAF | **ADM** | **Architecture Vision** | **B-D-A-T** | **O-AA** | **Enterprise Architecture** | **Architecture Artifacts** | **HLD** | **LLD** | **Preliminary Phase**

DEFINITIONS

TOGAF: The Open Group Architecture Framework — the most widely used framework for enterprise architecture. Published and maintained by The Open Group.

ADM (Architecture Development Method): The core of TOGAF. A cycle that guides the development of enterprise architecture. Ground principle: B-D-A-T (Business, Data, Applications, Technology).

Architecture Vision: The first phase in the ADM cycle that defines the high-level vision of what the architecture should achieve, feeding from architecture principles defined in the Preliminary Phase.

O-AA (Open Agile Architecture): A newer framework by The Open Group for agile architecture, focusing on business agility and customer experience as the heart of architecture.

EXPLANATION

Enterprise architecture for multi-cloud starts with TOGAF's ADM cycle. The cycle follows B-D-A-T: Business → Data → Applications → Technology.

The Preliminary Phase sets the framework and architecture principles, which feed into the Architecture Vision (Phase A), followed by Business Architecture (Phase B), Information Systems Architecture (Phase C), and Technology Architecture (Phase D).

Architecture artifacts follow a hierarchy: Enterprise Architecture → High-Level Design (HLD) → Low-Level Design (LLD).

IMPORTANT POINTS

- TOGAF ADM = standard framework; Preliminary Phase → Architecture Vision → B → C → D
- B-D-A-T: Business drives architecture; Technology is the last decision, not the first
- Architecture Vision (Phase A): sets high-level goals; Business Architecture (Phase B): defines business principles
- O-AA (2020): newer, agile framework placing customer experience at the heart of architecture

DIAGRAM AVAILABLE

(Refer Online: TOGAF ADM Cycle)

4. Defining Architecture Principles for Multi-Cloud

PYQ / EXPECTED QUESTIONS

- Discuss the role and application of architectural principles in managing multi-cloud architectures. [SEE 8a, 8M]
- Explain how architecture principles are defined from use cases in a multi-cloud environment? Describe at least five categories of principles. [CIE Improvement 2025, 10M]
- Illustrate the application of Application, Data, Business, and Technology Principles in real-world multi-cloud environments. [SEE 7b, 8M]

KEYWORDS

Architecture Principles | **Principle Groups** | **Durable** | **Consistent** | **SMART Goals** | **Cloud-Fit Strategy** | **Guiding Principles**

DEFINITIONS

Architecture Principles: Guidelines that enable the business to achieve its goals, support architectural decisions, and must be durable and consistent. Principles are derived from business use cases.

Business Use Case: Describes the actions a business must take in a specific, defined order to deliver a product or service to a customer that represents business value.

💡 EXPLANATION

The key to any preliminary phase is architecture principles — guidelines for fulfilling the architecture. Every principle must be challenged:

- Does the principle support the business goals?
- Is the principle clear (not subject to multiple interpretations)?
- Is the principle leading toward a clearly defined solution?

Principle groups for multi-cloud:

1. Business Principles
2. Security and Compliance Principles
3. Data Principles
4. Application Principles
5. Infrastructure and Technology Principles
6. Usability Principles
7. Process Principles

🔗 From the use case, the architect derives the principles, starting with the business principles.

5. Using Quality Attributes in Architecture

📋 PYQ / EXPECTED QUESTIONS

- Explain any four pillars of a Well-Architected Framework in the context of multi-cloud deployments. [CIE Improvement 2025, 5M]
- Quality attributes in architecture — expected as part of principles questions.

🔑 KEYWORDS

Continuous Architecture | Operability | Performance | Configurability | Discoverability | Security | Scalability | Robustness | Portability | Usability | RPO | RTO | Autoscaling

📖 DEFINITIONS

Quality Attributes: Values defined in the Continuous Architecture framework to which architecture must comply — since cloud architectures are fluid, dynamic, and must respond to changing customer demands.

RPO (Recovery Point Objective): Maximum allowed time for which data can be lost. Example: RPO of 1 hour means 1 hour of data loss is acceptable.

RTO (Recovery Time Objective): Maximum duration of downtime accepted by the business before systems must be restored.

Data Gravity: The phenomenon where large volumes of data attract applications toward the platform where the data resides, making migration difficult.

💡 EXPLANATION

Continuous Architecture defines the following quality attributes for cloud architecture:

- **Covers automation, monitoring, and logging. Ensures systems remain operational and secure. Includes end-to-end monitoring across all cloud platforms.**
- **Architecture must ensure systems respond within acceptable timeframes. End-to-end monitoring tracks performance from user perspective through to infrastructure.**
- **Systems in the cloud must be easy to configure. Configurations define desired state of a resource (CPU, memory, storage, security settings). Automation via DSC (Azure), Systems Manager (AWS), Compute Engine (GCP).**
- **Resources must be discoverable by other resources for communication and monitoring. Service discovery is critical in microservices — allows dynamic detection of service instances without hardcoding IP addresses.**

- **Focus on data protection. Design from angle of possible threats. Architecture safeguards data integrity. SecOps handles security policies and alerts.**
- **Scale-out (horizontal: add VMs), Scale-up (vertical: add CPU/memory/disk), Scale-down. Autoscaling must be triggered by defined thresholds. Consider scale units for VMs, databases, storage.**
- **Resilience, availability, retention, recovery. Design for availability at compute, application, and data layers. Uses RPO and RTO for backup and recovery design.**
- **Ability to move applications across cloud platforms without major disruption. Containers (Kubernetes) enable portability. Data gravity can limit portability.**
- **Applications must be accessible to users transparently. Requires DMZ design, jump boxes, connectivity planning. Technology should be completely transparent to end users.**

★ IMPORTANT POINTS

- Quality Attributes = architectural values from Continuous Architecture framework
- RPO and RTO must be defined per environment — critical systems need more stringent values
- Scale-out = horizontal (add VMs); Scale-up = vertical (add CPU/memory); Scale-down = reduce resources
- Data Gravity: large datasets attract applications — affects cloud strategy and portability
- All 9 quality attributes may be asked as definitions or short explanations in exams

6. Defining Principles from Use Cases

A business use case describes the workflow to produce and deliver a product/service. From the use case, the architect derives principles, starting with business principles. Use cases can include:

- Delivering products in new business models (SaaS)
- Faster time to market through rapid deployment of dev environments
- Analysis of big data using data lakes in the cloud
- Disaster recovery using the cloud

7. Business Principles

📋 PYQ / EXPECTED QUESTIONS

- Write any two business strategy objectives used in cloud principles. [SEE Aug 2025, 2M]
- Illustrate the application of Business Principles in real-world multi-cloud environments. [SEE 7b, 8M]

🔑 KEYWORDS

SMART Goals | Business Mission | Time to Market | Revenue | Customer-Centric | FinOps | Business Agility | Subscription Economy

📖 DEFINITIONS

Business Principles: Principles that drive the business goals and strategic decisions. They start with business units setting out their goals aligned to the business mission statement. They are prerequisites for any subsequent architectural stage.

SMART Goals: Specific, Measurable, Attainable, Relevant, and Timely goals. Example: 'Release of the web shop for product X in the North America region on June 1.'

💡 EXPLANATION

Business principles cover a wide variety of topics:

- Faster response to customers
- Faster deployment of new products (time to market)
- Improve quality of services or products

- Engage more with employees
- Real digital goals such as releasing a new website or web shop
- Increase revenue and profit while controlling costs (FinOps)

Business principles are defined in Phase B of the TOGAF ADM cycle (Business Architecture phase). They are prerequisites for all subsequent architectural stages.

★ IMPORTANT POINTS

- Business principles are listed FIRST — they drive all other principle groups
- SMART formulation required for all goals — measurable and time-bound
- Cloud-fit, not cloud-first: choose the right cloud for the right purpose

8. Security and Compliance Principles

📋 PYQ / EXPECTED QUESTIONS

- Discuss security and compliance principles in multi-cloud architecture. [Expected based on syllabus]

🔑 KEYWORDS

Zero Trust | **Security by Design** | **CIS Framework** | **GDPR** | **HSM** | **Micro-segmentation** | **MITRE ATT&CK** | **Encryption** | **Compliance** | **Privacy**

📖 DEFINITIONS

Zero Trust: Organizations do not trust anything within their networks and platforms. Every device, application, or user is monitored. Platforms are micro-segmented. Zero Trust is first and foremost a business principle — it assumes the organization has already been attacked.

Security by Design: Every component in the environment is designed to be secure from the architecture stage — built-in security. Platforms are hardened, code is encrypted/hashed, and security frameworks are applied from the start.

CIS Framework: Center for Internet Security — contains 20 critical security controls covering various sorts of attacks on different IT layers. Example: Control 13 = data protection (encryption at rest and in transit).

GDPR: General Data Protection Regulation — international privacy law. Compliance with GDPR is a common security and compliance principle in European enterprises.

💡 EXPLANATION

Security principles are the second most important group, right after business principles, because they impact every single aspect of the architecture.

Example principle: 'The architecture must comply with GDPR.' This drives: securing databases, encrypting data, controlling access with authentication and authorization.

In multi-cloud, data can be placed anywhere — PaaS and SaaS solutions used across clouds make compliance more challenging than in traditional data centers.

★ IMPORTANT POINTS

- Zero Trust = assume breach; micro-segment everything
- Security by Design = security is built into every layer, not added later
- CIS: 20 critical controls; not one-size-fits-all; organization must decide which apply
- Compliance principles include: GDPR, privacy laws, industry regulations, data residency requirements

9. Data Principles

📋 PYQ / EXPECTED QUESTIONS

- Mention any two differences between a system of record and a system of engagement. [CIE Improvement 2025, 2M]
- Illustrate the application of Data Principles in real-world multi-cloud environments. [SEE 7b, 8M]

KEYWORDS

Accuracy | **Relevance** | **Timeliness** | **Consistency** | **Privacy & Security** | **System of Record** | **System of Engagement** | **System of Intelligence** | **Data Lake** | **Data Gravity**

DEFINITIONS

System of Record: Data management or information storage systems that hold data. Includes databases, data lakes. Examples: ERP, CRM, CMS.

System of Engagement: Systems used to collect or access data. Includes email, collaboration platforms, mobile apps, IoT devices.

System of Intelligence: An emerging layer using AI/ML to analyze data from systems of record and engagement. Examples: Azure ML, AWS SageMaker, Google Cloud AI.

Data Lake: Large data stores holding mostly raw data, hosted in public clouds. Enables data analytics at petabyte scale.

EXPLANATION

The five most important data principles:

- **Data must be accurate, complete, and reliable. Inaccurate data leads to flawed decisions.**
- **Data must be relevant to the problem. Irrelevant data adds noise to analysis.**
- **Data must be up to date. Outdated data leads to incorrect conclusions.**
- **Data must be consistent in format, definitions, and units of measurement across sources.**
- **Data must be protected from unauthorized access. Sensitive data (personal, financial) must be secured.**

Azure: Data Factory, Databricks | AWS: EMR, Athena | GCP: BigQuery — for data analytics on data lakes.

IMPORTANT POINTS

- 5 Data Principles: Accuracy, Relevance, Timeliness, Consistency, Privacy & Security
- System of Record = storage (holds data) | System of Engagement = access (uses data)
- System of Intelligence = AI/ML layer on top of record and engagement systems (emerging)
- Data Gravity: huge datasets attract applications — migrating to another cloud becomes difficult

10. Application Principles

PYQ / EXPECTED QUESTIONS

- Illustrate the application of Application Principles in real-world multi-cloud environments. [SEE 7b, 8M]

KEYWORDS

Microservices | **Cloud-Native Apps** | **API** | **Data-Driven** | **Spewak** | **TOGAF Phase C** | **Modularity** | **Platform-Independent** | **Interoperability** | **Mobile-First**

DEFINITIONS

Application Principles: Principles that ensure applications make data accessible to the business and support business mission. Based on Spewak's enterprise architecture planning (1992) — business mission is the most important driver.

Cloud-Native Apps: Applications defined with roles and functions, built on code-based modularity and microservices. Communicate with other apps via APIs or triggers. Can be hosted anywhere.

Twelve-Factor App: A methodology for cloud-native apps (2005, still relevant). Three major infrastructure requirements: portability between platforms, no difference between dev and production, and supports scaling without major architecture changes.

💡 EXPLANATION

Application principles in multi-cloud (from TOGAF Phase C — Information Systems Architecture):

- Applications must safeguard data quality, make data accessible, and ensure data can be used
- Modern apps are cloud-native: microservices, APIs, modular, platform-independent
- Apps should be enabled for mobility, open standards, interoperability, and scalability
- Apps must be designed with extreme agility — customer demands change rapidly
- Apps must be well-documented, including runbooks, to support DevOps pipelines

★ IMPORTANT POINTS

- Spewak: Business mission → data-driven → applications → make data accessible
- Cloud-native = microservices + APIs + modular code + platform-independent
- Twelve-Factor App: portable, no dev-prod gap, scalable without architecture changes
- TOGAF Phase C = Information Systems Architecture (Data + Application)

11. Infrastructure and Technology Principles

📋 PYQ / EXPECTED QUESTIONS

- Illustrate the application of Technology Principles in real-world multi-cloud environments. [SEE 7b, 8M]

🔑 KEYWORDS

TOGAF Phase D | Technology Architecture | Landing Zone | Bleeding Edge | Micro-segmentation | IaC | Scalable | Twelve-Factor App | Network Topology

📖 DEFINITIONS

Infrastructure and Technology Principles: Generic guiding principles (not product-specific constraints) for the infrastructure layer. Defined in TOGAF Phase D (Technology Architecture). Covers platform location, network topology, infrastructure components, and system interdependencies.

Bleeding Edge Technology: A new, non-proven, experimental technology that imposes risk when deployed because it is still unstable and unreliable. A principle might state: avoid bleeding-edge technology in production.

💡 EXPLANATION

Infrastructure principles are generic and guiding, NOT a product catalog. Key principle examples:

- Infrastructure must be scalable (scale out, up, and down)
- Infrastructure must allow micro-segmentation for security
- Avoid bleeding-edge, experimental technologies in production environments
- Twelve-Factor App requirements: portable, code-based (IaC), scalable without major changes

In multi-cloud, Phase D involves: defining the landing zone, network topology across clouds, infrastructure components for specific apps/data stores, and system interdependencies.

★ IMPORTANT POINTS

- Principles = generic and guiding; NOT a product list or technology standard catalog
- TOGAF Phase D = Technology Architecture (platforms, networking, infrastructure)
- Micro-segmentation and scalability are core infrastructure principles in multi-cloud
- Twelve-Factor App drives infrastructure requirements since 2005 — still relevant

12. Principles for Processes

KEYWORDS

DevOps | Agile | SAAFe | Spotify Model | Automation | IaC | Code-Only Principle | Master Branch | Forking | CI/CD

DEFINITIONS

Process Principles: Principles concerned with deployment and automation processes in multi-cloud. These are about the way of working, not IT service management (ITSM) processes. Example: 'The organization embraces agility in its core processes (SAAFe or Spotify model).'

EXPLANATION

Process principles focus on:

- Automation as much as possible — define all manually-executed tasks in automated workflows
- Code-only principle — all changes must be made from the code base
- Forked code can only be committed back to master if tested in a separate (non-production) environment
- Working in DevOps/Agile — defined as a principle with consequences (sprint planning, epics, backlogs)

IMPORTANT POINTS

- Process principles focus on WAY OF WORKING — not ITSM processes
- Biggest pitfall: making process principles too complex and detailed (describe principle, not the process itself)
- A principle might be: 'Organization embraces agile in core processes (SAAFe/Spotify)' — not the sprint length

UNIT 5 — DevOps, DevSecOps, AIOps & GreenOps

13. Introducing DevOps and CI/CD

PYQ / EXPECTED QUESTIONS

- Briefly explain the DASA principles (Explain Any Five). [CIE-3 2024, 5M]
- Illustrate in detail the Build and Release pipeline used in DevOps. [CIE-3 2024, 6M]
- Develop a continuous integration and deployment pipeline plan. [SEE 9a, 8M]
- Identify the various steps involved in successful implementation of DevOps. [CIE-3 2024, 4M]

KEYWORDS

DASA | **CI/CD** | **Continuous Integration** | **Continuous Delivery** | **Continuous Deployment** | **DevOps Cycle** | **Feedback Loop** | **Build Pipeline** | **Release Pipeline** | **GTA** | **DoD**

DEFINITIONS

DevOps: A culture and new way of working based on a feedback loop for developing and operating IT systems. Defined by DASA (DevOps Agile Skills Association) with 6 principles.

Continuous Integration (CI): Built on the principle of a shared repository where code is frequently updated and shared. Allows developers to work together on the same code simultaneously. Changes are integrated and ready for testing.

Continuous Delivery: Putting artifacts (built in CI) into environments (development → staging → production), one after the other, with testing and approvals in between. Takes weeks or days.

Continuous Deployment: Code going directly to production from CI in a fully automated way without any human intervention. Changes can go to production several times per day.

CI/CD Pipeline: Automated pipeline enabling the DevOps cycle: continuous development, testing, and deployment. Combines all responsibilities from planning to management.

DASA Principles (6):

8. Customer-centric action: Develop with the customer in mind — what they need and expect in terms of functionality. Also the goal of domain-driven design.
9. Create with the end in mind: Envision how the application will look when completely finished.
10. End-to-end responsibility: Teams take responsibility from start to finish of the application life cycle. Mottos: 'you build it, you run it', 'you break it, you fix it', 'you destroy it, you rebuild it better.'
11. Cross-functional autonomous teams: Teams are able and allowed to make decisions themselves in the development process.
12. Continuous improvement: Constant improvement of the application, processes, people, and tools. DevOps is a culture, a mindset.
13. Automate as much as possible: The only way to gain speed in delivery and deployment. Automation also limits failures such as misconfigurations.

IMPORTANT POINTS

- DASA: 6 principles — Customer-centric, End in mind, E2E Responsibility, Autonomous teams, Continuous improvement, Automate
- CI = shared repo + frequent integration | CD = automated delivery to environments | Continuous Deployment = no human intervention
- Key difference: Continuous Delivery takes days/weeks; Continuous Deployment happens multiple times per day
- CI/CD enables DevOps cycle — all responsibilities lie with the team

DIAGRAM AVAILABLE

(Refer Online: DevOps CI/CD Lifecycle)

14. Getting Started with CI/CD

PYQ / EXPECTED QUESTIONS

- Identify the various steps involved in successful implementation of DevOps. [CIE-3 2024, 4M]
- Design a comprehensive build and release pipeline for the Swiggy mobile application. [SEE 10a, 8M]

KEYWORDS

GTA | **DoD** | **User Stories** | **IaC** | **Jenkins** | **Azure DevOps** | **GitHub Actions** | **Q&A Environment** | **Proof of Concept** | **Consistency**

DEFINITIONS

GTA (Generic Test Agreement): Describes what and how tests must be executed before systems are pushed to production — a quality and test plan.

DoD (Definition of Done): Describes the conditions and acceptance criteria a system must meet before deployment to production. Standard of quality for end product.

Build Pipeline: The pipeline where code is developed, built, and tested before being sent to the release pipeline.

Release Pipeline: The pipeline where built code is configured and released for production, tested in Q&A environment.

Steps for Successful DevOps Implementation:

14. Form autonomous teams with end-to-end responsibility. Teams work in short agile sprints (2–4 weeks). Authority to make decisions without excessive approvals.
15. Choose CI/CD system. Popular tools: Jenkins, Azure DevOps, GitHub Actions. Ensure ALL teams use the same system (consistency is key).
16. Perform proof of concept. Create automated process pipeline. Use GTA as starting point; define DoD with user stories (acceptance criteria, unit testing, code review, functional and technical tests passed).
17. Automate as much as possible. Use IaC (Infrastructure as Code). App code and infrastructure code in same repository, accessible by all teams.

Build and Release Pipeline workflow:

- Build Pipeline: Source Code → Build → Test → Deploy (to dev environment)
- Release Pipeline: Trigger → Build → Functional Test → Deploy Code → Integration Test → Deploy (to Q&A → Production)

DIAGRAM AVAILABLE

(Refer Online: Build and Release Pipeline)

★ IMPORTANT POINTS

- 4 Steps: Autonomous teams → Choose CI/CD → Proof of Concept → Automate
- DoD: unit testing done + code reviewed + acceptance criteria signed off + functional and technical tests passed
- Q&A environment: full build tested before releasing to production

15. Working under Version Control

KEYWORDS

Git | **Subversion** | **Version Control** | **Commit** | **Repository** | **Main Branch** | **IaC** | **Docker** | **Kubernetes** | **Terraform** | **CloudFormation** | **DSC**

DEFINITIONS

Version Control: Systems (Git, Subversion) that enable teams to organize source code, test scripts, deployment scripts, and configuration scripts. Enables retrieval of historical system states.

Commit: A code change along with metadata on who made the change and the rationale behind it. Ensures code is consistent, repeatable, and predictable.

EXPLANATION

Everything in the cloud is code — version control is essential for:

- Application code
- API scripts and references
- IaC components (VMs, network devices, storage, DNS, firewall rules)
- Infrastructure configuration packages
- Cloud templates: AWS CloudFormation, Azure DSC, Terraform files
- Container definitions: Docker files
- Container orchestration: Kubernetes, Docker Swarm files
- Test scripts

IMPORTANT POINTS

- Version control = consistent, repeatable, predictable deployments across multi-cloud
- All items — code, config, IaC, containers — must be under version control
- Consistent version control = rapid yet secure and reliable application deployments

16. Using Push and Pull Principles in CI

PYQ / EXPECTED QUESTIONS

- What is forking? With a suitable diagram, explain pushing code to the main branch using a fork. [CIE-3 2024, 5M]

KEYWORDS

Main Branch | **Mainline** | **Trunk** | **Commit** | **Push** | **Pull Request** | **Feature Branch** | **Fork** | **Merge** | **Forking**

DEFINITIONS

Main Branch (Mainline/Trunk): The primary code repository from which all developers work. In GCP, referred to as trunk. The single source of truth for code.

Commit: Pushing code changes back to the main branch. Done as frequently as possible in CI.

Pull Request: The stage where changed code is reviewed before being merged into the main branch.

Forking: Creating a copy (feature branch) from the main branch. Developers work in isolation on the forked code and commit back to main after intensive testing and a pull request.

Method 1: Pushing Code Directly to the Main Branch

Developers change small pieces of code and merge directly back to the main branch (commit). These commits are done multiple times per day.

- Advantage: Frequent releases; immediate feedback; no complex integrations
- Disadvantage: Automated tests after each commit are critical to avoid breaking the main
- Philosophy: 'You break it, you fix it' — developers are responsible for build, commit, and result

Method 2: Pushing Code to Forks of the Main (Feature Branch)

Teams copy code from main and create a separate or feature branch (forking). Work is done in isolation on the forked code.

- Used for: Major developments and new features

- Developers work in isolation; commit back to main after testing (via pull request)
- Downside: Can lead to complex integrations; NOT done frequently
- Intensive testing required before merging back to main branch
- In GCP context: referred to as feature-driven development (not trunk-based)

DIAGRAM AVAILABLE

(Refer Online: Developers merging code directly to the main branch)

(Refer Online: Developers working in a feature branch before merging to the main branch)

★ IMPORTANT POINTS

- Direct push = frequent commits + immediate integration | Forking = isolation + infrequent merging
- Pull Request = code review stage before merge. ALWAYS required before merging to main.
- Forking: Fork master → Fork feature branch → Commit changes → Merge after pull request → Version 1.1 on Main

17. Best Practices for Working with CI/CD

🔑 KEYWORDS

Four-Eyes Principle | **Pair Programming** | **Peer Review** | **Automated Test Scripts** | **Code Review** | **Pull Request**

📖 DEFINITIONS

Four-Eyes Principle: Code reviewed while being written by working in developer pairs, where the second developer reviews the first developer's code (pair programming). Also done as peer review at end of development.

Pair Programming: Two developers work together — the second developer reviews the first developer's code in real time. Implements the four-eyes principle.

💡 EXPLANATION

Biggest pitfall: code reviews that are too extensive — requiring approval from multiple stakeholders slows the DevOps process significantly.

Two best practices:

- Four-eyes principle (pair programming or peer review) — prevents errors before commit
- Automated test scripts — MUST run before code is committed to main branch to ensure systems keep functioning

★ IMPORTANT POINTS

- Two CI/CD best practices: Four-eyes principle + Automated test scripts
- Automated tests = first layer of DevSecOps security integration into DevOps
- Extensive code reviews = anti-pattern in DevOps (slows down the pipeline)

18. Using the DevSecOps Maturity Model (DSOMM)

📋 PYQ / EXPECTED QUESTIONS

- Explain DevSecOps Maturity Model levels. [Expected — high priority]

🔑 KEYWORDS

DevSecOps | **OWASP** | **DSOMM** | **Level 0-4** | **SAST** | **DAST** | **Security by Design** | **Security Culture** | **IaC** | **IAM** | **SIEM**

📖 DEFINITIONS

DevSecOps: The integration of security into the DevOps process at every stage — from development to deployment and operations. Security is not a sauce added on top; it is built in from the start.

OWASP DSOMM: Open Web Application Security Project DevSecOps Maturity Model — a framework to assess and improve software development and delivery practices with security integration. Available at: owasp.org/www-project-devsecops-maturity-model/

SAST: Static Application Security Testing — automated tool to scan code for vulnerabilities as it is being written.

DAST: Dynamic Application Security Testing — automated tool to test running applications for vulnerabilities.

DSOMM 5 Levels:

- **No formalized security program or processes. Security is not a consideration in software development.**
- **Security is taken into account but limited to a separate team/department. No collaboration between security and development teams.**
- **Security is integrated into the development process and teams collaborate. Security is NOT yet automated; testing not in every stage.**
- **Security integrated into EVERY stage. Security testing is automated (SAST/DAST), performed continuously throughout development.**
- **The DevOps process is continuously monitored and improved. Organization has a culture of security — security is part of every decision and action.**

How to use DSOMM:

18. Assess current security practices — identify gaps and current maturity level
19. Set SMART goals for security integration
20. Create a roadmap with specific milestones, timelines, and responsibilities
21. Implement security automation tools (SAST, DAST) — reaches Level 3
22. Integrate security into culture: design, dev, test, deploy, ops — reaches Level 4

★ IMPORTANT POINTS

- DSOMM: 5 levels (0–4): No security → Siloed → Integrated → Continuous → Continuous Improvement
- Level 3 = automated security testing (SAST/DAST) at every stage
- Level 4 = security culture — everyone is responsible, in every decision
- DevSecOps also includes infrastructure (IaC with IAM, network security, logging)
- IaC tools: AWS CloudFormation, Azure ARM, Google Cloud Deployment Manager

19. Managing Traceability and Auditability

🔑 KEYWORDS

Observability | **Traceability** | **Auditability** | **Prometheus** | **Grafana** | **Fluentd** | **Jaeger** | **OpenTelemetry** | **Audit Trail** | **Multi-Cloud Observability**

📖 DEFINITIONS

Observability: The ability to collect and analyze data from across the entire software stack (infrastructure, applications, user interactions) including multi-cloud settings. Enables identification of security threats and quick response.

Traceability: The ability to track and trace events and activities across the entire software system — following the flow of data and code to identify points of vulnerability.

Auditability: The ability to review and verify actions taken by various stakeholders in the software development and deployment process. Maintains a complete audit trail for accountability, transparency, and forensic analysis.

💡 EXPLANATION

Observability is essential in multi-cloud DevSecOps — organizations release multiple times per month, week, or even day across various clouds. Observability tools must be cloud-agnostic.

Popular multi-cloud observability tools:

- **Open-source monitoring** — collects and queries metrics from multiple cloud providers.
- **Open-source** — visualizes data; creates dashboards and alerts from multiple sources.
- **Open-source data collector** — collects, processes, and forwards log data from multiple clouds.
- **Open-source distributed tracing system** — traces requests across multiple services and clouds.
- **Vendor-neutral framework** — standard way to collect, process, and export telemetry data across clouds and languages.

★ IMPORTANT POINTS

- Traceability = track flow of code/data | Auditability = review actions of stakeholders
- Tools must be cloud-agnostic in multi-cloud: Prometheus, Grafana, Fluentd, Jaeger, OpenTelemetry
- Observability enables: security threat identification, compliance verification, incident response, forensic analysis

20. Automating Security Best Practices using Frameworks

🔑 KEYWORDS

OWASP Top 10 | **CIS Controls** | **MITRE ATT&CK** | **SIEM** | **TTPs** | **Chef** | **Puppet** | **Ansible** | **DeTTECT** | **SQL Injection** | **XSS** | **Configuration Management**

📖 DEFINITIONS

OWASP Top 10: A list of the most critical security risks to web applications. Used to automate security scanning in CI/CD pipelines.

CIS Controls: A set of 20 guidelines for securing information systems. Automated using configuration management tools (Chef, Puppet, Ansible). Available for each cloud: Azure, AWS, GCP, Alibaba, OCI.

MITRE ATT&CK: A knowledge base of Tactics, Techniques, and Processes (TTPs) used by attackers. Used to detect suspicious activity and automate incident response via SIEM tools.

SIEM: Security Information and Event Management — tools that detect suspicious activity and trigger automated incident response.

💡 EXPLANATION

The three major security frameworks for DevSecOps automation:

- **Automate vulnerability scanning (SQL injection, XSS) using SAST/DAST tools integrated into CI/CD. Also use automated code analysis tools during development.**
- **Use configuration management tools (Chef, Puppet, Ansible) to automatically configure security settings on servers, networks, and applications across the organization.**
- **Use SIEM tools to detect suspicious network/system activity. Automate incident response tools. Use DeTTECT to score data log quality, visibility, and detection coverage.**

Example: DeTTECT — originally invented for Rabobank (Dutch bank) — assists in using MITRE ATT&CK to score and compare log source quality, visibility, detection, and threat actor behavior.

★ IMPORTANT POINTS

- OWASP Top 10 → scan for web app vulnerabilities in CI/CD pipeline
- CIS Controls → enforce security policies via Chef/Puppet/Ansible config management

- MITRE ATT&CK → SIEM tools + automated incident response + DeTTECT tool
- All three frameworks can be automated and integrated into DevSecOps pipelines

21. Understanding the Concept of AIOps

PYQ / EXPECTED QUESTIONS

- What is AIOps? Explain the Components used in AIOps. [CIE-3 2024, 5M]
- AIOps — e-commerce firm scenario: how AIOps with data analytics generates predictive insights. [SEE 9b, 8M]
- Define GreenOps? Write its advantage. [SEE Aug 2025, 2M]

KEYWORDS

AIOps | **Artificial Intelligence for Operations** | **Data Analytics** | **Machine Learning** | **Automation** | **Visualization** | **Baseline** | **Anomaly Detection** | **NoOps** | **Noisy Neighbor** | **Root Cause Analysis**

DEFINITIONS

AIOps (Artificial Intelligence for Operations): Combines analytics of big data and ML to automatically investigate and remediate incidents in the IT environment. AIOps systems learn to correlate incidents across components by continuously analyzing logs and performance of assets.

NoOps (No IT Operations): The evolution beyond AIOps where all operational activities are fully automated — teams concentrate entirely on development. Requires engineers and governance to set up the baseline.

Noisy Neighbor: An application or system that disproportionately consumes shared resources, impacting neighboring systems. AIOps identifies and proactively manages noisy neighbors.

AIOps Components (4):

- **Gathers data from various sources** — log files, system metrics, monitoring data, social media posts, service desk incidents. Aggregates data, looks for trends and patterns, compares to known models.
- **Uses algorithms with a baseline representing normal behavior of systems, applications, and users.** Constantly evaluates new patterns and learns — teaches itself what new normal behavior is and what events create alerts. Prioritizes events and triggers remediation.
- **The HEART of AIOps.** Detects issues, unexpected changes, or abnormalities in behavior, then prioritizes and starts remediation without human intervention. Example: if a system runs out of memory, it automatically increases memory size. Can also predict peak usage and scale before it occurs.
- **Real-time dashboards and extensive reporting capabilities for engineers to have visibility of the system and its automated actions.** Helps improve architecture design. AIOps does NOT change architecture — architects are still needed.

★ IMPORTANT POINTS

- AIOps = Big Data Analytics + ML + Automation + Visualization
- Automation = heart of AIOps — detects and remediates WITHOUT human intervention
- ML establishes baseline, then continuously learns new patterns and anomalies
- AIOps is proactive and predictive; monitoring tools are descriptive and reactive
- Evolution: Monitoring → Log Analytics → Predictive (AIOps) → Prescriptive (advanced AIOps)

DIAGRAM AVAILABLE

(Refer Online: Evolution of monitoring to AIOps)

22. Optimizing Cloud Environments using AIOps

PYQ / EXPECTED QUESTIONS

- Illustrate how to optimize the cloud environments using AIOps. [CIE-3 2024, 5M]

KEYWORDS

Anomaly Detection | Capacity Optimization | Root Cause Analysis | Autoscaling | Prescriptive | Predictive | Standardize | Learning System | Data-Driven

EXPLANATION

Two major benefits of AIOps:

- Speed and accuracy in detecting anomalies and responding without human intervention
- Capacity optimization — cloud providers offer native scale-out/-up mechanisms; AIOps optimizes them since it knows thresholds dynamically (vs. hardcoded by engineers)

Additional AIOps optimization capabilities:

- **Analyzes data from multiple sources, correlates events, quickly diagnoses and resolves problems.**
- **Automates response to common issues — scale resources, restart failed services, reconfigure network settings. Reduces manual intervention and human error.**
- **Monitors environments, recommends optimizations based on usage patterns, identifies cost reduction opportunities.**
- **Gives architects better insight into connections between all applications, systems, network, and security devices.**

Guidelines for Successful AIOps Implementation:

- **Start small with a pilot — do NOT implement the full IT environment at once. Iterate from the pilot.**
- **Feed both IT data AND business data into AIOps. Business data (e.g., seasonal sales patterns) enables IT optimization based on business events. Businesses MUST be involved in implementation.**
- **Multi-cloud environments must be implemented consistently using IaC from one central repository. Consistent deployments help AIOps learn system behavior quickly and detect anomalies faster.**

IMPORTANT POINTS

- 3 AIOps guidelines: Start with a pilot | Feed business data | Standardize with IaC
- AIOps extends DevOps: predicts impact on systems when CI/CD deploys changes
- Root cause analysis: AIOps correlates events across clouds to find the true cause of issues
- Noisy neighbor: AIOps identifies, warns, and takes proactive measures across platforms

23. Exploring AIOps Tools for Multi-Cloud

PYQ / EXPECTED QUESTIONS

- Discuss the following cloud tools: AWS CloudWatch, GCP, OCI. [CIE-3 2024, 6M]

KEYWORDS

AWS CloudWatch | AWS X-Ray | GCP Cloud Operations | Stackdriver | OCI Oracle Management Cloud | Splunk | Dynatrace | IBM Cloud Pak | Moogsoft | AppDynamics

DEFINITIONS

AWS CloudWatch: AWS monitoring service for collecting metrics, logs, and events from AWS resources. Provides visibility into operational health.

AWS X-Ray: A distributed tracing service providing end-to-end visibility into application performance and behavior. Allows users to visualize and analyze the flow of requests across distributed systems. Identifies performance bottlenecks and errors in microservices.

GCP Cloud Operations (formerly Stackdriver): AIOps-enabled tool for GCP. Collects and analyzes metrics, logs, and traces from GCP resources and applications. Provides visibility into operational health and performance. Uses ML algorithms to detect anomalies and perform root cause analysis.

Oracle Management Cloud (OCI): Suite of management and monitoring services for Oracle Cloud Infrastructure. Includes log analytics, application performance monitoring, and infrastructure monitoring. Uses ML algorithms for anomaly detection and root cause analysis.

Splunk Enterprise: Cloud-agnostic AIOps platform that collects, correlates, and analyzes data from IT infrastructure, applications, and security systems. Has APIs for major cloud providers.

💡 EXPLANATION

All AIOps tools work in layers (Refer: Layers of AIOps diagram). AIOps market: ~\$26B in 2020, projected >\$600B by 2030.

Cloud-agnostic AIOps tools (important for multi-cloud):

- Dynatrace
- Splunk
- Cisco AppDynamics
- Moogsoft
- IBM Cloud Pak for Watson

Key selection criteria for multi-cloud AIOps: Must have APIs for major cloud providers and integrate with native monitoring solutions AND third-party tools.

★ IMPORTANT POINTS

- AWS: CloudWatch (metrics/logs) + X-Ray (distributed tracing, microservices)
- GCP: Cloud Operations (formerly Stackdriver) — ML-driven anomaly detection
- OCI: Oracle Management Cloud — log analytics + APM + infrastructure monitoring
- Multi-cloud AIOps tools: Splunk, Dynatrace, AppDynamics, Moogsoft, IBM Cloud Pak
- Key feature of all: auto-discover changes in real time and predict impact on other components

DIAGRAM AVAILABLE

(Refer Online: Layers of AIOps)

24. Introducing GreenOps

📋 PYQ / EXPECTED QUESTIONS

- What do you mean by GreenOps? Discuss its advantages. [CIE-3 2024, 4M]
- Define GreenOps? Write its advantage. [SEE Aug 2025, 2M]
- Describe a real-world scenario where Green AIOps is applied to enhance efficiency of data centers. [SEE 10b, 8M]

🔑 KEYWORDS

GreenOps | Carbon Footprint | Sustainability | Energy Consumption | Hyperscalers | Shared Resources | Auto-scaling | Environmental Sustainability | Energy Monitoring

📖 DEFINITIONS

GreenOps: The practice of using cloud technology to optimize the environmental sustainability of IT operations, helping organizations to reduce their carbon footprint and operate in a more environmentally friendly manner.

💡 EXPLANATION

GreenOps and AIOps are intensively related — both make use of AI. GreenOps uses AI and ML to optimize IT operations and reduce energy consumption.

Advantages of GreenOps / Cloud for Sustainability:

- **Cloud allows organizations to use shared infrastructure, reducing overall energy consumption. Virtualized infrastructure and shared resources reduce the number of physical servers needed.**
- **Organizations can scale operations up or down quickly based on demand. Avoids maintaining excess capacity — reduces energy consumption and energy bills.**
- **Repetitive tasks automated; ML algorithms identify areas for optimization (e.g., data center cooling optimization — reduces energy use and improves IT efficiency).**
- **Hyperscalers (Azure, AWS, GCP, OCI, Alibaba) offer energy monitoring and reporting tools to track usage and identify improvements. Cloud-based analytics tools optimize energy consumption.**
- **Optimizing IT operations through GreenOps directly reduces carbon footprint, saves money on energy bills, and contributes to global sustainability.**

Example: Machine learning algorithms can optimize data center cooling, reducing energy consumption and improving operational efficiency.

Example: Green AIOps: Reducing energy in data centers by automating workload scheduling to off-peak hours, scaling down idle resources, and optimizing cooling systems using predictive ML models.

★ IMPORTANT POINTS

- GreenOps = use cloud tech to optimize environmental sustainability of IT operations
- GreenOps + AIOps: both use AI/ML — AIOps optimizes performance; GreenOps optimizes sustainability
- Hyperscalers (AWS, Azure, GCP, OCI) offer energy monitoring tools
- Key: shared resources + dynamic scaling + automation = reduced energy consumption + lower carbon footprint

FINAL REVISION SECTION

Unit 4 Revision Sheet

23. Scaffold = foundation structure for cloud landing zone. Pillars: governance, identity, cost, security, operations.
24. Authentication (AD + Kerberos) + Authorization (RBAC/IAM) + Federation = identity management in multi-cloud.
25. TOGAF ADM: Preliminary Phase → Architecture Vision (A) → Business (B) → Info Systems (C) → Technology (D). B-D-A-T principle.
26. Architecture Principles: Business → Security & Compliance → Data → Application → Infrastructure → Usability → Processes.
27. Quality Attributes (Continuous Architecture): Operability, Performance, Configurability, Discoverability, Security, Scalability, Robustness, Portability, Usability.
28. Business Principles: SMART goals, faster time-to-market, revenue, customer-centric, cloud-fit strategy.
29. Security Principles: Zero Trust, Security by Design, CIS (20 controls), GDPR compliance, encryption.
30. Data Principles: Accuracy, Relevance, Timeliness, Consistency, Privacy & Security. Systems of Record vs. Engagement vs. Intelligence.
31. Application Principles: Spewak — business mission → data-driven → accessible apps. Cloud-native, microservices, API, Twelve-Factor App.
32. Infrastructure Principles: Generic + guiding (not product list). Scalable, micro-segmented, avoid bleeding-edge. TOGAF Phase D.
33. Process Principles: Automate everything, code-only principle, DevOps/Agile (SAFe/Spotify), master branch governance.

Unit 5 Revision Sheet

34. DASA 6 Principles: Customer-centric, End in mind, E2E responsibility, Autonomous teams, Continuous improvement, Automate.
35. CI = frequent code integration; CD = delivery to environments with approvals; Continuous Deployment = fully automated, no human.
36. 4 DevOps Steps: Autonomous teams → Choose CI/CD → Proof of concept (DoD/GTA) → Automate (IaC).
37. Version Control: Git/Subversion. ALL items under control: app code, IaC, Docker, K8s, test scripts, configs.
38. Direct Push: small changes to main, multiple times/day. Forking: copy main → feature branch → isolated dev → merge after testing.
39. CI/CD Best Practices: Four-eyes principle (pair programming) + automated test scripts before commit.
40. DSOMM: Level 0 (no security) → 1 (siloe) → 2 (integrated) → 3 (continuous, automated: SAST/DAST) → 4 (culture).
41. Traceability = track code/data flow | Auditability = verify stakeholder actions. Tools: Prometheus, Grafana, Fluentd, Jaeger, OpenTelemetry.
42. Security Frameworks: OWASP Top 10 (web app vulns, CI/CD), CIS Controls (config management), MITRE ATT&CK (TTPs, SIEM).
43. AIOps Components: Data Analytics + ML + Automation (heart) + Visualization.
44. AIOps Optimization: Anomaly detection, capacity optimization, root cause analysis, predictive scaling.

45. AIOps Tools: AWS (CloudWatch + X-Ray), GCP (Cloud Operations/Stackdriver), OCI (Oracle Management Cloud). Multi-cloud: Splunk, Dynatrace, AppDynamics.
46. GreenOps: Use cloud + AI to reduce carbon footprint. Benefits: shared resources, dynamic scaling, automation, energy monitoring.

Important Definitions (Exam-Ready)

Scaffold: Foundation structure to build and manage cloud environments in a landing zone.

Multi-Cloud: Use of two or more cloud computing systems simultaneously for redundancy and avoiding vendor lock-in.

RBAC: Role-Based Access Control — manages access to environment based on roles assigned to identities.

Zero Trust: Security principle: do not trust anything within networks; every device/user/app is monitored; micro-segmented.

TOGAF ADM: Architecture Development Method — TOGAF's core cycle for enterprise architecture. B-D-A-T principle.

Quality Attributes: Values from Continuous Architecture framework to which architecture must comply (Operability, Performance, Scalability, etc.).

RPO: Recovery Point Objective — maximum allowed time for which data can be lost.

RTO: Recovery Time Objective — maximum duration of downtime accepted by the business.

DevOps: Culture and way of working based on feedback loops for developing and operating IT systems (DASA 6 principles).

CI (Continuous Integration): Shared repository where code is frequently updated and integrated — allows teams to collaborate on code simultaneously.

CD (Continuous Delivery): Automated transfer of artifacts to environments (dev → staging → production) with testing and approvals. Takes days/weeks.

Continuous Deployment: Fully automated code to production — no human intervention. Multiple deployments per day.

Commit: A code change pushed to repository, with metadata on who changed it and why.

Forking: Creating a feature branch by copying from the main branch. Developers work in isolation; intensive testing before merging.

Pull Request: Code review stage before changes are merged into the main branch.

DSOMM: DevSecOps Maturity Model (OWASP) — 5 levels to assess and improve security integration into DevOps.

Observability: Ability to collect and analyze data from across entire software stack across all clouds.

Traceability: Ability to track and trace events and activities across the entire software system.

Auditability: Ability to review and verify actions of stakeholders in software development and deployment. Maintains audit trail.

AIOps: Artificial Intelligence for Operations — combines big data analytics and ML to automatically investigate and remediate IT incidents.

NoOps: No IT Operations — all operational activities fully automated; teams focus entirely on development.

GreenOps: Practice of using cloud technology to optimize environmental sustainability of IT operations and reduce carbon footprint.

Data Gravity: Large datasets attract applications toward the platform where data resides, making migration to another cloud difficult.

SAST: Static Application Security Testing — scans code for vulnerabilities as it is written.

DAST: Dynamic Application Security Testing — tests running applications for vulnerabilities.

DoD (Definition of Done): Describes conditions and acceptance criteria a system must meet before deployment to production.

Important Keywords

Unit 4 Keywords:

Scaffold | Landing Zone | AD | RBAC | IAM | Kerberos | LDAP | Zero Trust | TOGAF | ADM | B-D-A-T | Architecture Vision | Quality Attributes | Operability | Scalability | Robustness | Portability | RPO | RTO | Data Gravity | System of Record | System of Engagement | System of Intelligence | SMART Goals | CIS | GDPR | Security by Design | Micro-segmentation | Spewak | Microservices | Cloud-Native | Twelve-Factor App | Bleeding Edge | SAFe | Spotify Model | IaC | O-AA | FinOps

Unit 5 Keywords:

DASA | CI/CD | Continuous Integration | Continuous Delivery | Continuous Deployment | Git | Subversion | Commit | Main Branch | Fork | Pull Request | Pair Programming | Four-Eyes Principle | DSOMM | SAST | DAST | Observability | Traceability | Auditability | Prometheus | Grafana | Fluentd | Jaeger | OpenTelemetry | OWASP Top 10 | CIS Controls | MITRE ATT&CK | DeTTECT | SIEM | AIOps | Data Analytics | ML | Automation | Visualization | Root Cause Analysis | Noisy Neighbor | NoOps | X-Ray | Stackdriver | Splunk | Dynatrace | GreenOps | Carbon Footprint | Hyperscalers | DoD | GTA

Frequently Asked PYQ Concepts

From CIE-3 2024 (Most Likely to Repeat):

- DASA 6 Principles (5M) — Define all 6 with explanations
- Forking — definition + diagram + explanation (5M)
- AIOps — definition + 4 components (5M)
- AIOps Cloud Optimization — 3 guidelines + 2 major benefits (5M)
- AIOps Tools — AWS X-Ray, GCP Cloud Operations, OCI Oracle Management Cloud (6M)
- GreenOps — definition + advantages (4M)
- Build and Release Pipeline — diagram + explanation (6M)
- Steps for successful DevOps implementation (4M)

From SEE August 2025 (High Priority):

- Define Scaffold in multi-cloud context (2M)
- Define GreenOps + advantages (2M)
- Two business strategy objectives in cloud principles (2M)
- Two roles of Azure (2M)
- Architecture principles — discuss role + application in multi-cloud (8M)
- Application + Data + Business + Technology Principles in real-world (8M)
- CI/CD pipeline plan (8M)
- AIOps with data analytics for e-commerce — predictive insights (8M)
- Build and release pipeline for Swiggy mobile app (8M)
- Green AIOps in data center efficiency (8M)

From CIE Improvement 2025:

- Define Scaffold (2M)
- System of Record vs System of Engagement — 2 differences (2M)
- Architecture Vision in TOGAF ADM (2M)

- Architecture principles from use cases — 5 categories with examples (10M)
- Key business drivers for multi-cloud + TOGAF ADM role (5M)
- Four pillars of Well-Architected Framework (5M)

Last-Minute Revision (2–3 Pages)

UNIT 4 — Quick Recall

SCAFFOLD = Foundation for cloud landing zone. Treat cloud as a data center.

IDENTITY: AD (authentication, Kerberos) + RBAC/IAM (authorization) + Zero Trust (micro-segment, assume breach).

TOGAF ADM Cycle: Prelim Phase (principles) → A (Vision) → B (Business) → C (Info Systems: Data + Apps) → D (Technology). B-D-A-T.

7 PRINCIPLE GROUPS: Business → Security → Data → Application → Infrastructure → Usability → Processes.

9 QUALITY ATTRIBUTES: Operability | Performance | Configurability | Discoverability | Security | Scalability | Robustness | Portability | Usability.

DATA PRINCIPLES (5): Accuracy, Relevance, Timeliness, Consistency, Privacy & Security.

Systems: Record (stores data: ERP, CRM) | Engagement (accesses data: apps, IoT) | Intelligence (AI/ML layer).

Business Principles = FIRST. SMART goals. Cloud-fit, not cloud-first.

Security = Zero Trust + Security by Design + CIS 20 controls + GDPR compliance.

Infrastructure = generic principles: scalable, micro-segmented, no bleeding-edge, IaC, Twelve-Factor App.

Process Principles = automate everything, code-only principle, agile (SAFe/Spotify).

UNIT 5 — Quick Recall

DASA 6 PRINCIPLES: (1) Customer-centric (2) End in mind (3) E2E responsibility (4) Autonomous teams (5) Continuous improvement (6) Automate.

CI = shared repo + frequent integration. Continuous Delivery = to environments with approvals.

Continuous Deployment = fully automated to prod.

4 DevOps STEPS: Autonomous teams → CI/CD system → PoC (DoD/GTA) → Automate (IaC).

VERSION CONTROL: All code under Git/Subversion: app code, IaC, Docker, K8s, test scripts, configs, terraform.

DIRECT PUSH: multiple commits/day, small changes. FORKING: copy → feature branch → isolated dev → pull request → merge. More testing required.

CI/CD BEST PRACTICES: Four-eyes (pair programming/peer review) + automated test scripts before commit.

DSOMM 5 LEVELS: 0 (no security) → 1 (siloes) → 2 (integrated) → 3 (automated: SAST/DAST) → 4 (security culture).

OBSERVABILITY TOOLS: Prometheus (metrics), Grafana (dashboards), Fluentd (logs), Jaeger (tracing), OpenTelemetry (vendor-neutral).

SECURITY FRAMEWORKS: OWASP (web vulns in CI/CD), CIS (config management: Chef/Puppet/Ansible), MITRE ATT&CK (TTPs, SIEM, DeTTECT).

AIOPS COMPONENTS: Data Analytics + ML (baseline, learn, prioritize) + Automation (HEART, remediation) + Visualization (dashboards).

AIOPS OPTIMIZATION: Anomaly detection | Capacity optimization | Root cause analysis | Prescriptive recommendations.

AIOPS GUIDELINES: (1) Start small — pilot (2) Feed business data (3) Standardize with IaC.

AIOPS TOOLS: AWS = CloudWatch + X-Ray (distributed tracing). GCP = Cloud Operations/Stackdriver (ML-driven). OCI = Oracle Management Cloud. Multi-cloud = Splunk, Dynatrace, AppDynamics, Moogsoft, IBM.

GREENOPS: Use cloud + AI to reduce carbon footprint. Advantages: shared resources + dynamic scaling + automation + energy monitoring. Related to AIOps (both use AI).

— End of Examination Notes — Units 4 & 5 —